

EDS LAB

ASSIGNMENT:

To accept an object mass in kilograms and velocity in meters per second and display its momentum. Momentum is calculated as $e=mc^2$ where m is the mass of the object and c is its velocity.

ANS:

```
mass = float(input("Enter mass in kilograms: "))  
  
velocity = float(input("Enter velocity in meters per second: "))  
  
momentum = mass * velocity  
  
print(f"Momentum: {momentum:.2f} kg·m/s")
```

2) Write a Python program for following conditions.

If n is single digit print square of it.

If n is two digit print square root of it.

If n is three digit print cube root of it.

3. Read the birth date and salary in rupees of employees. Perform data transformation for birthdate

to age and also salary which is in rupees to salary in dollars using functions.

4. Print the reverse number of a given number,

ANS:

```

import math
from datetime import datetime

# --- Task 1: Number-based operations ---
n = int(input("Enter a number: "))

if 0 <= n <= 9: # single digit
    print("Square:", n ** 2)
elif 10 <= n <= 99: # two digit
    print("Square root:", math.sqrt(n))
elif 100 <= n <= 999: # three digit
    print("Cube root:", round(n ** (1/3), 2))
else:
    print("Number is not single, double, or triple digit.")

# --- Task 2: Employee data transformation ---
birthdate_str = input("Enter birthdate (YYYY-MM-DD): ")
salary_rupees = float(input("Enter salary in rupees: "))

# Convert birthdate to age
birthdate = datetime.strptime(birthdate_str, "%Y-%m-%d")
today = datetime.today()
age = today.year - birthdate.year - ((today.month, today.day) <
(birthdate.month, birthdate.day))

# Convert salary to dollars (approx rate: 1 USD = 83 INR)
salary_dollars = salary_rupees / 83

print("Age:", age, "years")
print(f"Salary in dollars: {salary_dollars:.2f} USD")

# --- Task 3: Reverse a number ---
num = int(input("Enter a number to reverse: "))
reverse_num = int(str(num)[::-1])
print("Reversed number:", reverse_num)

```

3) Write a Python program to print different patterns

ANS

```
# Program to print different patterns
```

```
# Pattern 1: Right-angled triangle
```

```
print("Pattern 1: Right-angled triangle")
```

```
for i in range(1, 6):
```

```
    print("*" * i)
```

```
# Pattern 2: Pyramid
```

```
print("\nPattern 2: Pyramid")
```

```
for i in range(1, 6):
```

```
    print(" " * (6 - i) + "*" * (2 * i - 1))
```

```
# Pattern 3: Inverted triangle
```

```
print("\nPattern 3: Inverted triangle")
```

```
for i in range(5, 0, -1):
```

```
    print("*" * i)
```

4)Select the number from the entered list and find its position in Python (use Linear Search).

ANS

```
def linear_search(arr, target):
```

```
    for i in range(len(arr)):
```

```
        if arr[i] == target:
```

```
            return i # return position (index)
```

```
    return -1 # if not found
```

```
# Input list from user
```

```
numbers = list(map(int, input("Enter numbers separated by space: ").split()))
```

```
# Input target number
```

```
target = int(input("Enter the number to search: "))
```

```
# Perform linear search
position = linear_search(numbers, target)

# Display result
if position != -1:
    print(f"Number {target} found at position {position} (index starts
at 0).")
else:
    print(f"Number {target} not found in the list.")
```

5) Choose cricket team of eleven players find the captain of the team (consider tallest person as a captain) using dictionary.

ANS

```
# Program to choose cricket team captain based on height

# Create a dictionary of players and their heights (in cm)
team = {
    "Rohit": 175,
    "Virat": 178,
    "Hardik": 183,
    "Jadeja": 180,
    "Bumrah": 185,
    "Gill": 176,
    "Surya": 177,
    "Ishan": 172,
    "KL Rahul": 179,
    "Shami": 182,
    "Kuldeep": 174
}

# Find the tallest player (captain)
captain = max(team, key=team.get)
```

```
# Display the team and captain
print("Cricket Team Players and Heights:")
for player, height in team.items():
    print(f"{player}: {height} cm")

print("\nCaptain of the team:", captain)
print("Height:", team[captain], "cm")
```

6) Prepare/Take dataset for any real life application. Read a dataset into an array. Perform following operations on it as:

- Perform all matrix operations
- Horizontal and vertical stacking of Numpy Arrays
- Custom sequence generation
- Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators
- Copying and viewing arrays
- Data Stacking, Searching, Sorting, Counting, Broadcasting

ANS

```
import numpy as np
```

```
# --- Step 1: Create a dataset (real-life example: student marks in 3 subjects) ---
```

```
students = np.array([
    [85, 90, 88],
    [78, 82, 80],
    [92, 95, 94],
    [70, 75, 72],
    [88, 85, 90]
])
```

```
print("Dataset (Student Marks):\n", students)
```

```
# --- Step 2: Matrix Operations ---
```

```
print("\nMatrix Addition:\n", students + 5)
```

```

print("\nMatrix Subtraction:\n", students - 5)
print("\nMatrix Multiplication:\n", students * 2)
print("\nMatrix Transpose:\n", students.T)

# --- Step 3: Horizontal and Vertical Stacking ---
extra_marks = np.array([[5, 5, 5], [3, 3, 3], [4, 4, 4], [2, 2, 2],
[6, 6, 6]])
h_stack = np.hstack((students, extra_marks))
v_stack = np.vstack((students, extra_marks))
print("\nHorizontal Stack:\n", h_stack)
print("\nVertical Stack:\n", v_stack)

# --- Step 4: Custom Sequence Generation ---
seq = np.arange(10, 60, 10)
print("\nCustom Sequence (10 to 50 step 10):", seq)

# --- Step 5: Arithmetic and Statistical Operations ---
print("\nSum of all marks:", np.sum(students))
print("Mean:", np.mean(students))
print("Maximum:", np.max(students))
print("Minimum:", np.min(students))
print("Standard Deviation:", np.std(students))

# --- Step 6: Mathematical Operations ---
print("\nSquare Root of Marks:\n", np.sqrt(students))
print("Exponential of Marks:\n", np.exp(students / 100))

# --- Step 7: Bitwise Operators ---
a = np.array([10, 20, 30])
b = np.array([5, 15, 25])
print("\nBitwise AND:", np.bitwise_and(a, b))
print("Bitwise OR:", np.bitwise_or(a, b))

# --- Step 8: Copying and Viewing Arrays ---
copy_arr = students.copy()
view_arr = students.view()
print("\nCopied Array:\n", copy_arr)
print("Viewed Array:\n", view_arr)

```

```
# --- Step 9: Data Stacking, Searching, Sorting, Counting,
Broadcasting ---
print("\nSorted Marks:\n", np.sort(students))
print("Search for value 90:", np.where(students == 90))
print("Count of values > 85:", np.count_nonzero(students > 85))

# Broadcasting example: add 5 bonus marks to all students
bonus = np.array([5, 5, 5])
print("\nBroadcasting (Add Bonus Marks):\n", students + bonus)
```

7) For Titanic dataset, Perform data analysis. Identify 5 grains for a given dataset. Perform a data

visualization for those grains using the Matplotlib library. (Use any 5 different graphs with proper

title, legends, axis names, etc.to map identified grains)

ANS

```
import pandas as pd
import matplotlib.pyplot as plt

# --- Step 1: Read the Titanic dataset ---
# You can download 'titanic.csv' from Kaggle or use seaborn's built-in
dataset
df = pd.read_csv("titanic.csv")

# --- Step 2: Display basic info ---
print("Dataset Overview:\n", df.head())
print("\nSummary Statistics:\n", df.describe())

# --- Step 3: Identify 5 grains (data insights) ---
# 1 Survival rate by gender
gender_survival = df.groupby('Sex')['Survived'].mean()
```

```

# 2 Survival rate by passenger class
class_survival = df.groupby('Pclass')['Survived'].mean()

# 3 Age distribution of passengers
age_distribution = df['Age'].dropna()

# 4 Number of passengers by embarkation port
embark_counts = df['Embarked'].value_counts()

# 5 Correlation between fare and survival
fare_survival = df.groupby('Survived')['Fare'].mean()

# --- Step 4: Data Visualization using Matplotlib ---

# Graph 1: Survival rate by gender (Bar Chart)
plt.figure(figsize=(6,4))
gender_survival.plot(kind='bar', color=['skyblue', 'lightcoral'])
plt.title("Survival Rate by Gender")
plt.xlabel("Gender")
plt.ylabel("Survival Rate")
plt.legend(["Survival Rate"])
plt.show()

# Graph 2: Survival rate by passenger class (Bar Chart)
plt.figure(figsize=(6,4))
class_survival.plot(kind='bar', color='lightgreen')
plt.title("Survival Rate by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Survival Rate")
plt.legend(["Survival Rate"])
plt.show()

# Graph 3: Age distribution (Histogram)
plt.figure(figsize=(6,4))
plt.hist(age_distribution, bins=20, color='gold', edgecolor='black')
plt.title("Age Distribution of Passengers")
plt.xlabel("Age")
plt.ylabel("Number of Passengers")

```

```
plt.show()
```

```
# Graph 4: Passengers by Embarkation Port (Pie Chart)
```

```
plt.figure(figsize=(6,6))
```

```
embark_counts.plot(kind='pie', autopct='%1.1f%%', startangle=90,  
colors=['lightblue','lightgreen','lightpink'])
```

```
plt.title("Passengers by Embarkation Port")
```

```
plt.ylabel("")
```

```
plt.show()
```

```
# Graph 5: Average Fare vs Survival (Line Chart)
```

```
plt.figure(figsize=(6,4))
```

```
fare_survival.plot(kind='line', marker='o', color='purple')
```

```
plt.title("Average Fare vs Survival")
```

```
plt.xlabel("Survived (0 = No, 1 = Yes)")
```

```
plt.ylabel("Average Fare")
```

```
plt.legend(["Average Fare"])
```

```
plt.show()
```